



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Python

Python programming Revisions

What is python



- Python is a high-level, interpreted, general-purpose programming language that is widely used in a variety of fields such as web development, automation, data science, artificial intelligence, systems automation, and more.
- Created by Guido van Rossum and released in 1991, the language is known for its simple and clear syntax, which makes it easy to read and write code.
- **Key features of Python:**
 - **Simple and legible syntax:** Compared to other languages, Python has a syntax that resembles English, making it easy to learn and use.
 - **Interpreted:** Python code runs line by line, which allows for quick testing and easy debugging.
 - **Multi-paradigm:** Supports multiple programming paradigms such as object-oriented, functional, and imperative.
 - **Extensive standard library:** Python offers a vast collection of ready-made modules and libraries, which makes it easy to develop complex applications.
 - **Portability:** Python is cross-platform, meaning the same code can run on different operating systems, such as Windows, macOS, and Linux.
 - **Active community:** Python has a large and active community, which translates into a lot of documentation, tutorials, libraries, and support.

What is python



- **What can Python do?**

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

- **Why Python?**

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Running Python code



There are two main ways to execute Python code:

1. Interactive Mode (REPL - Read-Eval-Print Loop)

- In interactive mode, you run Python code directly in the terminal or in an interactive environment, such as IDLE, Jupyter Notebook, or Python shell. In this mode, you can enter commands line by line, and the interpreter executes each line as soon as it is entered, returning the results immediately.
- Terminal/Python shell:
 - Open a terminal (or command prompt) and type `python` or `python3` (depending on your installation).
 - The Python interpreter will open, and you can directly type in the commands.
 - This mode is useful for testing small snippets of code or experimenting with functionality quickly.

```
>>> print("Hello, World!")
Hello, World!
>>> 2 + 3
5
```

Running Python code



2. Script Execution Mode

- In this form, you write the code in a file with the extension .py and then run the entire file at once, instead of line by line.
- This method is best suited for developing larger, complex programs, such as web applications, automated scripts, or complete software projects.
- How it works:
 - Create a file with Python code, for example, meu_programa.py.
 - In the terminal or command prompt, navigate to the directory where the file is saved and run the python file.
 - The code in the file will be executed and the result displayed in the terminal.



```
# meu_programa.py
print("Hello, World!")
resultado = 2 + 3
print(f"O resultado é {resultado}")
```

```
python meu_programa.py
```

```
Hello, World!
O resultado é 5
```

Python Indentation

- Indentation refers to the spaces at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code.
- The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.
- You have to use the same number of spaces in the same block of code, otherwise Python will give you an error.

```
if 5 > 2:  
    print("Five is greater than two!")
```

```
if 5 > 2:  
print("Five is greater than two!")
```

```
if 5 > 2:  
    print("Five is greater than two!")  
if 5 > 2:  
    print("Five is greater than two!")
```

```
if 5 > 2:  
| print("Five is greater than two!")  
| ← print("Five is greater than two!")
```

Using REPL Mode



- Python's REPL (Read-Eval-Print Loop) mode allows you to interact directly with the interpreter, typing commands line by line and receiving responses immediately.
- Here's a step-by-step guide on how to use REPL mode:
 - Open a terminal and type `python` or `python3`. Press Enter, and you'll enter Python's interactive mode.
 - In REPL mode, you can write and execute Python commands right away. The interpreter will read the input, evaluate the command, print the result, and then wait for the next input.
 - REPL mode also allows you to access built-in Python documentation.
 - To exit REPL mode, simply type `exit()` or CTRL+D.

```
C:\Windows\System32\cmd.e  x  +  v
C:\>python
Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep  6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> 2 + 3
5
>>> 5 * 10
50
>>> x = 10
>>> y = 20
>>> x+y
30
>>> len("Python")
6
>>> max(1, 5, 3, 9)
9
>>> help(len)
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.

>>> exit()
C:\>|
```

Variables and basic data types in Python



- Variables are one of the fundamental building blocks of programs written in Python. Variables hold data in memory.
- They have names, and they can be referenced by those names.
- Variables also have types, which specify what type of data they can store (such as string and integer), and they can be used in expressions that use operators (such as + and -) to manipulate their values.
- In Python, a variable is declared and assigned a value using the assignment operator =.
- The variable is on the left-hand side of the operator, and the value it is assigned on the right side.
- The right side can be an expression such as 2 + 2 and can even include other variables.
- **Reassignment:** Since Python is dynamically typed, you can change the value (and type) of a variable at any time during the program.
- **Updating values:** Variables can be updated using mathematical and other operations on their current values.

example_01.py > ...

```
1
2 x = 1      # assign variable x the value 1
3 y = x + 5   # assign variable y the value of x plus 5
4 z = y      # assign variable z the value of y
5
6 print(z)
```

P:\>python example_01.py

6

P:\>|

```
x = 10      # x is an integer
x = "ten"   # now x is a string
```

```
count = 0
count = count + 1 # Updates count to 1
```

```
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)
```

```
x = y = z = "Orange"
print(x)
print(y)
print(z)
```


Variable naming rules

- Variables must follow certain rules:
- They must start with a letter (a-z, A-Z) or an underscore (_).
- They cannot start with numbers.
- They can contain letters, numbers, and underscores, but no other special characters.
- Python is case-sensitive, so `age` and `Age` are different variables.

```
x = 10      # x is an integer (int)
y = "Hello" # y is a string (str)
z = 3.14    # z is a floating-point number (float)
```

```
age = 25
name = "Alice"
```

```
name = "John"
_age = 30
monthly_salary = 2500.00
```

Common types of variables in Python



- Integer (int): Whole numbers, like 10, -5, 0.
- Float (float): Decimal numbers, like 3.14, -0.5.
- String (str): Sequences of characters, like "Hello", "Python".
- Boolean (bool): True (True) or False (False) values.
- **Lists**, tuples, dictionaries: Data structures that store collections of values.

```
age = 25          # int
height = 1.75     # float
name = "Carlos"   # str
active = True     # bool
```

```
3  name = "Ana"
4  age = 28
5  height = 1.65
6
7  print(f"Name: {name}, Age: {age}, Height: {height}m")
8
```

```
C:\Windows\System32\cmd.e  X  +  v

P:\>python example_01.py
Name: Ana, Age: 28, Height: 1.65m

P:\>|
```

Built-in Data Types



- In programming, data type is an important concept.
- Variables can store data of different types, and different types can do different things.
- Python has the following data types built-in by default, in these categories:
 - Text Type: str
 - Numeric Types: int, float, complex
 - Sequence Types: list, tuple, range
 - Mapping Type: dict
 - Set Types: set, frozenset
 - Boolean Type: bool
 - Binary Types: bytes, bytearray, memoryview
 - None Type: NoneType

Setting data types



- If you set the assignments below, the intended data type is assumed:

Example	Data Type
<code>x = "Hello World"</code>	str
<code>x = 20</code>	int
<code>x = 20.5</code>	float
<code>x = 1j</code>	complex
<code>x = ["apple", "banana", "cherry"]</code>	list
<code>x = ("apple", "banana", "cherry")</code>	tuple
<code>x = range(6)</code>	range
<code>x = {"name" : "John", "age" : 36}</code>	dict
<code>x = {"apple", "banana", "cherry"}</code>	set
<code>x = frozenset({"apple", "banana", "cherry"})</code>	frozenset
<code>x = True</code>	bool
<code>x = b"Hello"</code>	bytes
<code>x = bytearray(5)</code>	bytearray
<code>x = memoryview(bytes(5))</code>	memoryview
<code>x = None</code>	NoneType

Working with numbers



- Most programs manipulate numbers. Computers treat integer numbers and decimal numbers differently.
- Python creates integers from a built-in data type called int, and decimals (floating-point numbers) as instances of float.
- Python's built-in type() function returns a variable's data type.
- The addition of the .0 to the end of 1 makes a large difference in how the programming language treats a value.
- The data type impacts how the value is stored in memory, how the processor (CPU) handles the data when evaluating expressions, how the data relates to other data, and what kinds of operations you can perform with it.

```
x = 1          # integer
x = 1.0        # decimal (floating point)
```

```
2 x = 1
3 print(type(x)) # outputs: <class 'int'>
4
5 x = 1.0
6 print(type(x)) # outputs: <class 'float'>
```

A screenshot of a Windows command prompt window. The title bar shows "C:\Windows\System32\cmd.e" with a close button and window controls. The command prompt shows the execution of a Python script: "P:\>python example_01.py". The output of the script is displayed on two lines: "<class 'int'>" and "<class 'float'>". The prompt "P:\>" is shown again on the next line.

```
C:\Windows\System32\cmd.e
P:\>python example_01.py
<class 'int'>
<class 'float'>
P:\>
```

Working with booleans

- Another common data type is the Boolean type, which holds the value True or False.
- Internally, bool is treated as a special type of integer. Technically, True has a value of 1 and False has a value of 0.
- Typically, Booleans aren't used to perform mathematical operations; rather, they're used to make decisions and perform branching.
- Nevertheless, it's interesting to understand the relationship between types.
- Many types are nothing more than specialized versions of more general types.
- Integers are a subset of floating-point numbers, and Booleans are a subset of integers.

```
x = True  
print(type(x)) # outputs: <class 'bool'>
```

Working with strings

- Along with numbers, strings are among the most commonly used data types.
- A string is a collection of zero or more characters.
- Strings are commonly declared using single quotation marks, but you can also use double quotation marks.
- You can add strings to other strings—an operation known as "concatenation"—with the same + operator that adds two numbers.

```
x = 'This is a string'
print(x) # outputs: This is a string
print(type(x)) # outputs: <class 'str'>
y = "This is also a string"
```

```
x = 'Hello' + ' ' + 'World!'
print(x) # outputs: Hello World!
```

Reading keyboard input

- Many programs are interactive. Supporting interactivity means you have a program that runs differently depending on the input. The person inputting data to a program is usually a user, but it can be another program. There are many ways to send input to a program; two common ways are via a graphical interface or a console.
- User input:
 - For reading input from the keyboard, Python provides the `input()` function. `input()` reads what the user types on the keyboard and returns it as a string. The example combines `input()` and `print()` to capture a person's name and display it on the screen.

```
name = input('Enter your name:')  
print(name)
```

```
print('What is your name?')  
name = input()  
print(name)
```


Reading keyboard input

- Reading numbers as input:
 - The input function always returns the typed value as a string (text).
 - This choice makes sense because the user can enter whatever value they like.
 - Even if the input is a valid number, it's still returned as a string type from the input function.
- Converting numbers to strings:
 - You can go the other direction as well.
 - The str() method will take an integer or float value and turn it into a string.
 - Calling the str() method is needed if you want the below code example to work.
 - The conversion ensures the integer, in its string form, is concatenated to the string on the left.

```
x = input('Enter a number: ')\nprint(type(x))
```

```
x = int(input('Enter a number: '))\nprint(type(x))
```

```
x = 5\nprint('The number is ' + str(x))
```

Example: build a calculator



- In this exercise, you'll learn to build a calculator. A calculator takes input and presents the result on the screen.

```
first_number = int(input('Type the first number: ')) ;\
second_number = int(input('Type the second number: ')) ;\
print("The sum is: ", first_number + second_number)
```

Python Arithmetic Operators



- Python divides the operators in the following groups:
 - Arithmetic operators
 - Assignment operators
 - Comparison operators
 - Logical operators
 - Identity operators
 - Membership operators
 - Bitwise operators

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Python Assignment Operators



- Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3
:=	print(x := 3)	x = 3 print(x)

Python If ... Else

- Python supports the usual logical conditions from mathematics:
 - Equals: `a == b`
 - Not Equals: `a != b`
 - Less than: `a < b`
 - Less than or equal to: `a <= b`
 - Greater than: `a > b`
 - Greater than or equal to: `a >= b`
- These conditions can be used in several ways, most commonly in "if statements" and loops.
- An "if statement" is written by using the if keyword.
- The **elif** keyword is Python's way of saying "if the previous conditions were not true, then try this condition".
- The else keyword catches anything which isn't caught by the preceding conditions.
- You can have if statements inside if statements, this is called nested if statements.

```
a = 33
b = 200
if b > a:
    print("b is greater than a")
```

```
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

```
x = 41

if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```



And, or, not, ... operators

- The and keyword is a logical operator and is used to combine conditional statements.
- The or keyword is a logical operator, and is used to combine conditional statements
- The not keyword is a logical operator, and is used to reverse the result of the conditional statement.

Test if `a` is greater than `b`, AND if `c` is greater than `a` :

```
a = 200
b = 33
c = 500
if a > b and c > a:
    print("Both conditions are True")
```

```
a = 200
b = 33
c = 500
if a > b or a > c:
    print("At least one of the conditions is True")
```

```
a = 33
b = 200
if not a > b:
    print("a is NOT greater than b")
```

Python While Loops



- With the while loop we can execute a set of statements as long as a condition is true. We need to increment i, or else the loop will continue forever.
- The while loop requires relevant variables to be ready. In the example, we need to define an indexing variable, i, which we set to 1.
- With the **break** statement we can stop the loop even if the while condition is true.
- With the **continue** statement we can stop the current iteration, and continue with the next.

Print i as long as i is less than 6:

```
i = 1
while i < 6:
    print(i)
    i += 1
```

```
1
2
3
4
5
```

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
1
2
3
```

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
1
2
4
5
6
```

Python For Loops

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.
- With the **for** loop we can execute a set of statements, once for each item in a list, tuple, set etc.
- Even strings are **iterable** objects, they contain a sequence of characters
- With the **break** statement we can stop the loop before it has looped through all the items.
- With the continue statement we can stop the current iteration of the loop, and continue with the next.
- A **nested loop** is a loop inside a loop. The "inner loop" will be executed one time for each iteration of the "outer loop"



```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

```
for x in "banana":
    print(x)
```

Exit the loop when **x** is "banana":

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

apple

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

apple
cherry

Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry

Python Functions



- A function is a block of code that only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.
- In Python a function is defined using the `def` keyword.
- To call a function, use the function name followed by parenthesis.
- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.
- The following example has a function with one argument (`fname`). When the function is called, we pass along a first name, which is used inside the function to print the full name

```
def my_function():  
    print("Hello from a function")  
  
my_function()
```

```
def my_function(fname):  
    print(fname + " Refsnes")  
  
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```



```
P:\>python example_01.py  
Emil Refsnes  
Tobias Refsnes  
Linus Refsnes  
  
P:\>|
```

Python Lambda

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments but can only have one expression.
- Syntax:
 - *lambda arguments : expression*
- The expression is executed, and the result is returned.
- Lambda functions can take any number of arguments.
- (Also, see nested lambda functions).

Add 10 to argument *a* , and return the result:

```
x = lambda a : a + 10  
print(x(5))
```

Multiply argument *a* with argument *b* and return the result:

```
x = lambda a, b : a * b  
print(x(5, 6))
```

Summarize argument *a* , *b* , and *c* and return the result:

```
x = lambda a, b, c : a + b + c  
print(x(5, 6, 2))
```

Exercise: factorial

- Write a Python function called `calculate_factorial(n)` that takes an integer `n` as input and returns the factorial of that number. The factorial of a non-negative integer `n` is the product of all positive integers less than or equal to `n`.
- The function should return:
 - if `n` is 0 (since $0! = 1$ by definition).
 - The factorial value if `n` is a positive integer.
- Definition:
 - Factorial of a number `n` is represented by $n!$.
 - Formula:
- $$n! = n * (n-1) * (n-2) * \dots * 1, \text{ for } n > 0$$
- Example:
 - $5! = 5 * 4 * 3 * 2 * 1 = 120$
 - $3! = 3 * 2 * 1 = 6$
 - $0! = 1$ (by definition)

Resolution: factorial



- In the first approach, we use a for loop to calculate the factorial by multiplying numbers from 1 to n.
- In the second approach, we define the function to call itself recursively to calculate the factorial.

```
def calculate_factorial(n):  
    # Start with 1 because the factorial of 0 and 1 is 1  
    result = 1  
  
    # If n is greater than 0, multiply result by each number up to n  
    for i in range(1, n + 1):  
        result *= i  
  
    return result  
  
# Example test cases  
print(calculate_factorial(5))  
print(calculate_factorial(3))  
print(calculate_factorial(0))
```

```
def calculate_factorial(n):  
    # Base case: if n is 0, return 1 (because 0! = 1)  
    if n == 0:  
        return 1  
  
    # Recursive case: multiply n by the factorial of (n - 1)  
    else:  
        return n * calculate_factorial(n - 1)  
  
# Example test cases  
print(calculate_factorial(5))  
print(calculate_factorial(3))  
print(calculate_factorial(0))
```

Exercise



- Convert Celsius to Fahrenheit
- Write a function `celsius_to_fahrenheit(c)` that converts a temperature from Celsius to Fahrenheit using the formula:
$$F = (C * 9/5) + 32.$$
- Don't forget the parenthesis for setting operation precedence, as in mathematics.
- Write a function that does the reverse conversion.

```
print( celsius_to_fahrenheit(25) )  
print('-----')  
print( fahrenheit_to_celcius(77) )
```

```
P:\>python example_01.py  
77.0  
-----  
25.0  
P:\>
```

Exercise



- Write a function `multiplication_table(n)` that prints the multiplication table for a given number `n` (from 1 to 10).

```
P:\>python example_01.py
1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
4 x 5 = 20
5 x 5 = 25
6 x 5 = 30
7 x 5 = 35
8 x 5 = 40
9 x 5 = 45
```

```
def multiplication_table(n):
    for i in range(10):
        print(f"{i+1} x {n} = {(i+1)*n}")
```

```
def multiplication_table(n):
    for i in range(1, 10):
        print(f"{i} x {n} = {i*n}")

multiplication_table(5)
```

Calculate the square root of a number



- It is equivalent to power the number to 0.5.
- The first example is for positive numbers.
- The second example is of complex numbers.

```
num = 8

# To take the input from the user
#num = float(input('Enter a number: '))

num_sqrt = num ** 0.5
print('The square root of %0.3f is %0.3f'%(num ,num_sqrt))
```

```
import cmath

num = 1+2j
# To take input from the user
#num = eval(input('Enter a number: '))

num_sqrt = cmath.sqrt(num)
print('The square root of {0} is {1:0.3f}+{2:0.3f}j'.format(num ,num_sqrt.real
, num_sqrt.imag))
```

Calculate the area of a triangle



- If a , b and c are three sides of a triangle. Then,
 - $s = (a+b+c)/2$
 - $\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$

```
# Python Program to find the area of triangle

a = 5
b = 6
c = 7

# Uncomment below to take inputs from the user
# a = float(input('Enter first side: '))
# b = float(input('Enter second side: '))
# c = float(input('Enter third side: '))

# calculate the semi-perimeter
s = (a + b + c) / 2

# calculate the area
area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
print('The area of the triangle is %0.2f' %area)
```


Python Program to Solve Quadratic Equation



- The standard form of a quadratic equation is:
 - $ax^2 + bx + c = 0$, where
 - a , b and c are real numbers and
 - $a \neq 0$
- The solutions of this quadratic equation is given by:
 - $$\frac{-b \pm (b^2 - 4ac)^{0.5}}{2a}$$
- Based on the previous example, modify the example to obtain the coefficients from the user input.
- Create a function named solveQuadratic(a,b,c) using the code provided.
- Ask for inputs from the user and call the function.
- Implement a while loop to calculate solutions for given inputs. The loop should stop when option="exit".

```
# Solve the quadratic equation ax**2 + bx + c = 0

# import complex math module
import cmath

a = 1
b = 5
c = 6

# calculate the discriminant
d = (b**2) - (4*a*c)

# find two solutions
sol1 = (-b-cmath.sqrt(d))/(2*a)
sol2 = (-b+cmath.sqrt(d))/(2*a)

print('The solution are {0} and {1}'.format(sol1,sol2))
```

Python Program to Check Prime Number



- A positive integer greater than 1 which has no other factors except 1 and the number itself is called a prime number.
- The numbers 2, 3, 5, 7 etc. are prime numbers as they do not have any other factors.
- But 6 is not prime (it is composite) since, $2 \times 3 = 6$.
- Using the code provided:
 - develop a function named `is_prime(number)`.
 - create a loop while(true) inside the function main.
 - Inside the loop ask for user input numbers.
 - If the provided number is zero, then break the loop, else call `is_prime()` and tell about the result,

```
# Program to check if a number is prime or not

num = 29

# To take input from the user
#num = int(input("Enter a number: "))

# define a flag variable
flag = False

if num == 0 or num == 1:
    print(num, "is not a prime number")
elif num > 1:
    # check for factors
    for i in range(2, num):
        if (num % i) == 0:
            # if factor is found, set flag to True
            flag = True
            # break out of loop
            break

# check if flag is True
if flag:
    print(num, "is not a prime number")
else:
    print(num, "is a prime number")
```

Exercises

- Write a function `sum_two_numbers(a, b)` that takes two numbers and returns their sum.
- Write a function `is_even(n)` that takes an integer and returns True if the number is even and False if it's odd.
- Write a function `find_largest(a, b, c)` that takes three numbers and returns the largest among them.
- Write a function `print_numbers(n)` that prints the numbers from 1 to n (inclusive) using a loop.
- Write a function `sum_to_n(n)` that returns the sum of all numbers from 1 to n.
- Write a function `count_vowels(s)` that takes a string s and returns the number of vowels (a, e, i, o, u) in the string.
- Write a function `find_max(lst)` that takes a list of numbers and returns the maximum value in the list.
- Write a function `fibonacci(n)` that returns a list of the first n Fibonacci numbers.
- Write a function `second_largest(lst)` that returns the second largest number in a list.
- Write a function `average(lst)` that returns the average of all the numbers in a list.



```
sum_two_numbers(5, 3) # Output: 8
```

```
is_even(4) # Output: True  
is_even(7) # Output: False
```

```
find_largest(3, 7, 5) # Output: 7
```

```
sum_to_n(5) # Output: 15 (1 + 2 + 3 + 4 + 5)
```

```
count_vowels("hello world") # Output: 3
```

```
find_max([1, 5, 3, 9, 2]) # Output: 9
```

```
fibonacci(5) # Output: [0, 1, 1, 2, 3]
```

```
second_largest([1, 5, 3, 9, 2]) # Output: 5
```

```
average([10, 20, 30, 40, 50]) # Output: 30
```

```
print_numbers(5)  
# Output:  
# 1  
# 2  
# 3  
# 4  
# 5
```

More exercises



- `sort_list([3,1,4,2,10,5])`
- `list_to_set([1,1,4,2,6,7,2,6])` # [1, 4, 2, 7, 6]
- A function which uses the previous ones to transform from list to set and provided the list sorted.
- What do you think would perform faster?
 - Sort then turn into set?
 - Turn in to set and sort?

Install matplotlib



- python -m ensurepip --upgrade
- pip install numpy
- pip install matplotlib

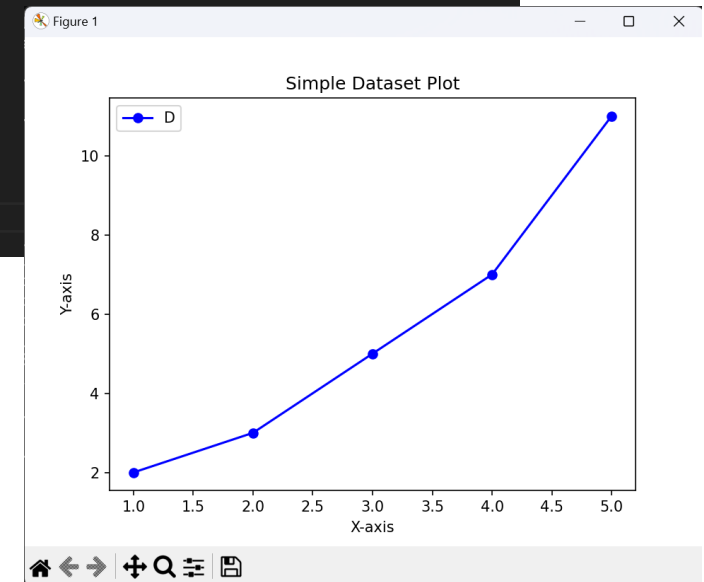
A screenshot of a Windows Command Prompt window. The title bar shows "C:\Windows\System32\cmd.exe" and "Command Prompt". The command prompt shows the command `C:\>pip install matplotlib` being entered. Below the command, several lines of output are displayed, indicating that the requirement for matplotlib is already satisfied and listing other installed packages and their versions.

```
C:\>pip install matplotlib
Requirement already satisfied: matplotlib in c:\python312\lib\site-packages
Requirement already satisfied: contourpy>=1.0.1 in c:\python312\lib\site-pac
Requirement already satisfied: cyclor>=0.10 in c:\python312\lib\site-package
Requirement already satisfied: fonttools>=4.22.0 in c:\python312\lib\site-pa
Requirement already satisfied: kiwisolver>=1.3.1 in c:\python312\lib\site-pa
Requirement already satisfied: numpy>=1.23 in c:\python312\lib\site-packages
```

Dataset plot

- The example uses the library **matplotlib** to create a basic line plot with labeled axes, a title, and a legend.
- Matplotlib is a popular Python library for data visualization, widely used for creating static, interactive, and animated plots.
- It provides a set of tools to generate a wide variety of plots, such as line plots, scatter plots, histograms, bar charts, and more.
- The library's core module, pyplot, provides a high-level interface that simplifies plot creation, enabling quick and easy visualization of data.

```
plot_data.py > ...
1  import matplotlib.pyplot as plt
2
3  # Sample dataset
4  x = [1, 2, 3, 4, 5]
5  y = [2, 3, 5, 7, 11] # Example: y = f(x), following a simple pattern
6
7  # Plotting the data
8  plt.plot(x, y, label="Sample Data", marker='o', linestyle='-', color='b')
9
10 # Adding labels and title
11 plt.xlabel("X-axis")
12 plt.ylabel("Y-axis")
13 plt.title("Simple Dataset Plot")
14
15 # Adding a legend
16 plt.legend("Data sample")
17
18 # Displaying the plot
19 plt.show()
```

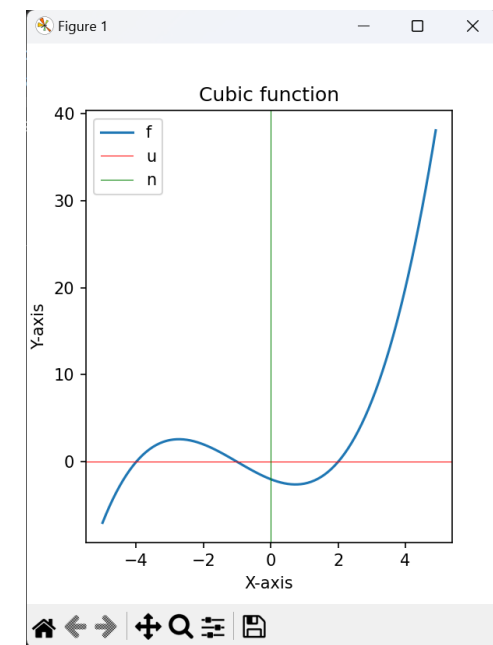


<https://matplotlib.org/stable/tutorials/pyplot.html>

Plot functions

- Plot ($y = a.x^3 + b.x^2 + c.x + d$) cubic functions.

```
plot_function.py > ...
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  x = np.arange(-5, 5, 0.1)
5  a = 1/4
6  b = 3/4
7  c = -6/4
8  d = -2
9  y = a*(x**3) + b*(x**2) + c*x + d
10
11 print('Values of x: ', x)
12 print('Values of y: ', y)
13
14 plt.plot(x, y)
15 # Draw X and Y axes at the origin
16 plt.axhline(0, color='red', linewidth=0.5)
17 plt.axvline(0, color='green', linewidth=0.5)
18
19 plt.xlabel("X-axis")
20 plt.ylabel("Y-axis")
21 plt.title("Cubic function")
22
23 # Adding a legend
24 plt.legend("function plot")
25
26 # Displaying the plot
27 plt.show()
28
29
30
31 plt.plot(x, y)
```



Python classes



- In Python, a class is a blueprint for creating objects that share similar attributes and behaviors.
- Classes allows defining the structure and functions (known as methods) that belong to the objects created from them.
- Classes help organize and encapsulate data and functionality, making code modular and easier to manage.
- A **class** typically consists of:
 - **Attributes** (variables) that hold data specific to the object.
 - **Methods** (functions) that define behaviors or actions that objects of the class can perform
- Each object created from a class is known as an **instance**, and it can have its own unique data while sharing the same structure and behavior defined by the class.

```
Person.py > ...
1 class Person:
2     # Constructor method to initialize attributes
3     def __init__(self, name, age):
4         self.name = name # Instance attribute
5         self.age = age   # Instance attribute
6
7     # Method to display person's info
8     def introduce(self):
9         print(f"Hello, my name is {self.name} and I am {self.age} years old.")
10
11 # Creating instances of the Person class
12 person1 = Person("Alice", 30)
13 person2 = Person("Bob", 25)
14
15 # Calling the method on each instance
16 person1.introduce()
17 person2.introduce()
```

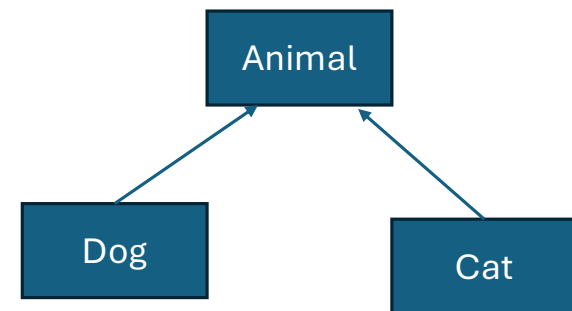
```
C:\Windows\System32\cmd.e  x  +  v
C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>python Person.py
Hello, my name is Alice and I am 30 years old.
Hello, my name is Bob and I am 25 years old.

C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>
```


Class inheritance



- **Inheritance** is an object-oriented programming concept in Python that allows a class (called the child or subclass) to inherit attributes and methods from another class (called the parent or superclass).
- This helps promote code reuse, as the child class can use the functionality of the parent class without needing to rewrite the code.
- Additionally, the child class can add its own unique methods and attributes or override methods from the parent class.
- For example, if we have a general class called **Animal**, we can define specific types of animals (like **Dog** and **Cat**) as subclasses that inherit from **Animal**.
- This way, we can create shared attributes and behaviors in the **Animal** class and then add or modify behaviors specific to **Dog** or **Cat**.



Class inheritance example



- Here is an exemple.

```
inheritance.py > ...
1  # Base class (Parent)
2  class Animal:
3      def __init__(self, name, age):
4          self.name = name
5          self.age = age
6
7      def make_sound(self):
8          print("This animal makes a sound.")
9
10     def info(self):
11         print(f"I am an animal named {self.name} and I am {self.age} years old.")
12
13     # Subclass (Child) for Dog
14     class Dog(Animal):
15         def make_sound(self):
16             print("Woof! Woof!")
17
18         def fetch(self):
19             print(f"{self.name} is fetching the ball.")
20
21     # Subclass (Child) for Cat
22     class Cat(Animal):
23         def make_sound(self):
24             print("Meow! Meow!")
25
26         def scratch(self):
27             print(f"{self.name} is scratching the furniture.")
```

```
28
29     # Creating instances of Dog and Cat
30     dog = Dog("Buddy", 5)
31     cat = Cat("Whiskers", 3)
32
33     # Calling methods on Dog instance
34     dog.info()          # Inherited from Animal
35     dog.make_sound()    # Overridden in Dog
36     dog.fetch()         # Unique to Dog
37
38     # Calling methods on Cat instance
39     cat.info()          # Inherited from Animal
40     cat.make_sound()    # Overridden in Cat
41     cat.scratch()       # Unique to Cat
42
```

```
C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>python inheritance.py
I am an animal named Buddy and I am 5 years old.
Woof! Woof!
Buddy is fetching the ball.
I am an animal named Whiskers and I am 3 years old.
Meow! Meow!
Whiskers is scratching the furniture.

C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>
```

Python modules



- A Python module is a file that contains Python code, typically defining functions, classes, and variables.
- Modules help organize code into logical sections, making it easier to maintain, reuse, and share.
- By splitting functionality into separate modules, we can keep our codebase organized and avoid duplicating code across projects.
- Modules can be standard library modules, third-party modules, or user-defined modules:
 - **Standard library modules:** Python includes a large number of built-in modules, such as `math`, `datetime`, and `os`, which provide useful functionality for mathematical operations, date and time handling, file manipulation, and more.
 - **Third-party modules:** These are external modules that you can install via package managers like `pip`, such as `requests` for HTTP requests or `numpy` for numerical computations.
 - **User-defined modules:** These are custom modules you create by saving Python code in a `.py` file, which can then be imported and used in other scripts. To use a module, you use the `import` statement, which allows you to access its functions, classes, and variables.
- **Standard library** and **third-party modules** provide useful pre-written functionality, while **user-defined modules** allow for custom functionality specific to your project.
- **Importing** a module allows us to use its functions, classes, and variables in other scripts.

Python modules



- Example:

```
math_operations.py > ...
1  # math_operations.py
2
3  def add(a, b):
4      return a + b
5
6  def subtract(a, b):
7      return a - b
8
9  def multiply(a, b):
10     return a * b
11
12 def divide(a, b):
13     if b != 0:
14         return a / b
15     else:
16         return "Cannot divide by zero"
17
```

```
main_script.py > ...
1  # main_script.py
2
3  import math_operations
4
5  # Using functions from the math_operations module
6  result_add = math_operations.add(5, 3)
7  result_subtract = math_operations.subtract(10, 4)
8  result_multiply = math_operations.multiply(6, 7)
9  result_divide = math_operations.divide(8, 2)
10
11 print("Addition:", result_add)           # Output: Addition: 8
12 print("Subtraction:", result_subtract)  # Output: Subtraction: 6
13 print("Multiplication:", result_multiply) # Output: Multiplication: 42
14 print("Division:", result_divide)       # Output: Division: 4.0
15
```

Threads



- In Python, **threads** are a way to enable a program to run multiple tasks concurrently, making it more efficient when performing operations that can occur in parallel.
- This concept is particularly useful when a program needs to handle multiple tasks that might otherwise block one another, such as downloading multiple files, handling user input, or managing different I/O operations simultaneously.
- How Threads Work in Python:
 - Threads are separate, lightweight units of execution within a single process. Each thread runs independently and shares the same memory space with other threads in the process.
 - In Python, the threading module provides the tools needed to create and manage threads.
- Use Cases for Python Threads:
 - **I/O-bound tasks:** Threads are especially useful for tasks where the program has to wait, such as reading or writing to files, waiting for network responses, or handling multiple user interactions.
 - **Background tasks:** Threads allow certain tasks to run in the background while the main program continues executing, which is ideal for updating the user interface, logging, or monitoring system resources in real-time.

Threads_example.py



- Printing numbers and letters in parallel.

```
Threads_example.py > ...
1
2 import threading
3 import time
4
5 # Function for the first thread
6 def print_numbers():
7     for i in range(5):
8         print(f"Number: {i}")
9         time.sleep(1)
10
11 # Function for the second thread
12 def print_letters():
13     for letter in 'ABCDE':
14         print(f"Letter: {letter}")
15         time.sleep(1)
16
17 # Create threads
18 thread1 = threading.Thread(target=print_numbers)
19 thread2 = threading.Thread(target=print_letters)
20
21 # Start threads
22 thread1.start()
23 thread2.start()
24
25 # Wait for both threads to complete
26 thread1.join()
27 thread2.join()
28
29 print("Both threads have finished execution.")
30
```

```
C:\Windows\System32\cmd.e  X  +  v
C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>python Threads_example.py
Number: 0
Letter: A
Number: 1
Letter: B
Number: 2
Letter: C
Number: 3
Letter: D
Number: 4
Letter: E
Both threads have finished execution.
C:\OneDrive\OneDrive - FCT NOVA\docencia\Robotica\week_06>
```

UI design in python



- UI (User Interface) design in Python involves creating graphical interfaces that allow users to interact with a Python program through buttons, input fields, menus, and other visual elements.
- Python provides several libraries to design UIs, making it possible to develop both simple and complex desktop applications. Some of the most commonly used libraries for UI design in Python are Tkinter, PyQt, Kivy, and wxPython.
- Key Concepts in UI Design:
 - **Widgets:** The building blocks of a UI, such as buttons, labels, text fields, sliders, and combo boxes. Widgets are used to display information and accept user input.
 - **Event Handling:** UIs are event-driven, meaning they respond to user actions like clicks, keystrokes, or selections. Event handlers are functions that define what happens when a user interacts with a widget.
 - **Layout Management:** The arrangement of widgets on the screen. Most UI libraries offer different layout options (such as grids, boxes, or absolute positioning) to organize widgets in a structured manner.
 - **Styling:** Visual customization of the UI to improve aesthetics and usability. This may involve changing fonts, colors, sizes, or even creating custom themes.
- **Tkinter** is the standard Python library for creating GUIs and is included with Python. It's simple to use and ideal for small to medium-sized applications.
- Others are PyQt, Kivy, wxPython, ...

UI exemple 1



- Simple UI

```
ui_example_1.py > ...
1  import tkinter as tk
2
3  # Initialize the main window
4  root = tk.Tk()
5  root.title("Simple UI")
6
7  # Create a label
8  label = tk.Label(root, text="Hello, Robotics!")
9  label.pack()
10
11 # Create a button
12 def on_button_click():
13     label.config(text="Button Clicked!")
14
15 button = tk.Button(root, text="Click Me", command=on_button_click)
16 button.pack()
17
18 # Run the main event loop
19 root.mainloop()
```

